

UNIT 2 – Manual Testing Techniques & Practices

Exhaustive Study Notes covering Testing Types, Black-Box Design, Smoke/Sanity, Regression & RTM

1. Manual Testing Overview

Manual Testing is the process of manually executing test cases without using automation scripts. Testers simulate end-user interactions to identify bugs and usability flaws.

Testing Approaches: • Black-Box Testing: Testing software without knowing internal code structure. • White-Box Testing: Testing internal code, logic paths, and statements (Unit testing). • Grey-Box Testing: Combination of both; testing with partial internal knowledge.

Software Testing Taxonomy (Static vs Dynamic, Functional vs Non-Functional)

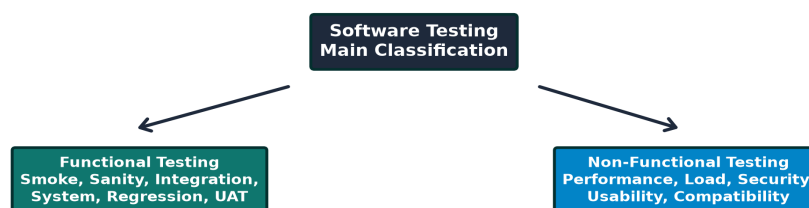


Diagram: Software Testing Taxonomy (Functional vs Non-Functional).

2. Functional Testing Design Techniques

Black-Box Test Design Techniques: 1. Equivalence Partitioning (EP): Divides input data into valid and invalid partitions. Test one representative value from each partition. 2. Boundary Value Analysis (BVA): Tests values at the boundaries of valid/invalid partitions (Min, Min-1, Min+1, Max, Max-1, Max+1). 3. Decision Table Testing: Tabular representation of complex business rules combining multiple inputs and resulting actions. 4. State Transition Testing: Tests system responses to state changes triggered by events.

Example: Age Input Field (Valid range 18 to 60)

Equivalence Partitions:

- Invalid: Age < 18 (e.g. 15)
- Valid: 18 <= Age <= 60 (e.g. 30)
- Invalid: Age > 60 (e.g. 70)

Boundary Values to Test:

- 17 (Min-1 -> Invalid)
- 18 (Min -> Valid)
- 19 (Min+1 -> Valid)
- 59 (Max-1 -> Valid)
- 60 (Max -> Valid)
- 61 (Max+1 -> Invalid)

Equivalence Partitioning (EP) & Boundary Value Analysis (BVA)

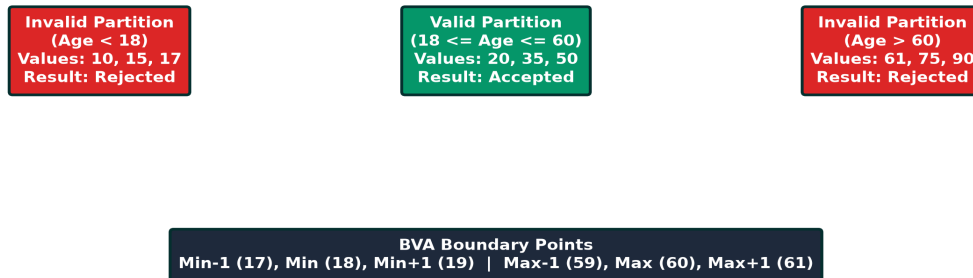


Diagram: Equivalence Partitioning (EP) & Boundary Value Analysis (BVA) Diagram.

3. Smoke Testing vs Sanity Testing

- **Smoke Testing (Build Acceptance Testing):** Wide and shallow testing executed on initial build to verify critical core functionalities before detailed testing.
- **Sanity Testing:** Narrow and deep testing executed on a stable build following bug fixes to verify specific functionality changes without breaking existing modules.

■ **Key Takeaway:** Smoke testing verifies overall build stability; Sanity testing verifies specific bug fix correctness.

4. Regression Testing & Re-Testing

- **Re-Testing:** Executing a failed test case again after developer fixes the defect to verify the fix.
- **Regression Testing:** Re-executing unchanged parts of the application to ensure recent code fixes haven't broken existing functionality.

Regression Types: 1. Unit Regression: Testing code within the same module. 2. Regional Regression: Testing modules linked to modified code. 3. Full Regression: Complete test suite execution prior to major release.

5. Test Case Writing Practice

Writing effective, unambiguous test cases requires clear steps and verifiable expected outcomes.

```
# Test Case: User Registration
TC ID: TC_REG_002
Title: Registration with already existing email
Precondition: User 'test@example.com' is already registered in DB
Steps:
  1. Open registration form
  2. Input Name 'John Doe'
  3. Input Email 'test@example.com'
  4. Input Password 'Pass1234!'
  5. Click 'Register'
Expected Output: Error message 'Email already registered' displayed. Account creation fails.
```

6. Defect Reporting & Logging

A Defect Report communicates bug details clearly to developers to ensure quick resolution.

```
# Sample Defect Report
Defect ID: BUG_AUTH_042
Title: 500 Internal Server Error when submitting empty login form
Severity: High | Priority: P1-Immediate
Environment: Staging (Chrome 126 / Windows 11)
Steps to Reproduce:
  1. Go to https://staging.example.com/login
  2. Leave Username and Password blank
  3. Click 'Sign In'
Expected Result: Inline validation message 'Username is required'
Actual Result: Page crashes with HTTP 500 Internal Server Error
Attachments: error_log_trace.png
```

7. Test Management & RTM

Test Management encompasses test planning, execution monitoring, bug tracking, and traceability.

Requirements Traceability Matrix (RTM): A grid document linking requirements to test scenarios, test cases, and defects.

Ensures 100% test coverage for all user requirements.

■ **Key Takeaway:** RTM ensures no functional requirement is missed during test execution.

Requirements Traceability Matrix (RTM) Architecture Flow



Diagram: Requirements Traceability Matrix (RTM) Architecture Flow.